

高级语言程序设计

实验报告

南开大学 计算机大类

姓名：李志翔

学号：2512218

班级：软件学院 2 班

2026 年 5 月 13 日

目录

一、 作业题目	3
二、 开发软件	3
三、 课题要求	3
四、 主要流程	4
1. 整体架构	4
2. 各模块实现	4
(1) 音频引擎模块 (AudioEngine)	4
(2) 可视化绘制模块 (Visualizer)	6
(3) 节奏游戏模块 (RhythmGame)	8
(4) 主窗口控制模块 (RhythmFlow)	9
3. 核心算法	9
(1) FFT 频谱分析算法	9
(2) 鼓点检测算法	10
(3) 粒子物理系统核心公式	10
(4) 连击加速机制	11
五、 测试与验证	12
六、 收获与总结	14
1. 技术收获	14
2. AI 说明	15
3. 总述	16

高级语言程序设计大作业实验报告

一、作业题目

RhythmFlow —— 音频驱动可视化与节奏游戏

本项目是一款基于 C++ 和 Qt6 开发的赛博朋克风格桌面应用。程序能够实时将音乐转化为动态频谱和粒子星云，并内置自动谱面生成的节奏游戏模式，同时兼具音乐播放器功能。用户可以通过鼠标与粒子交互，通过键盘控制播放和游戏，实现视听一体化的沉浸体验。

二、开发软件

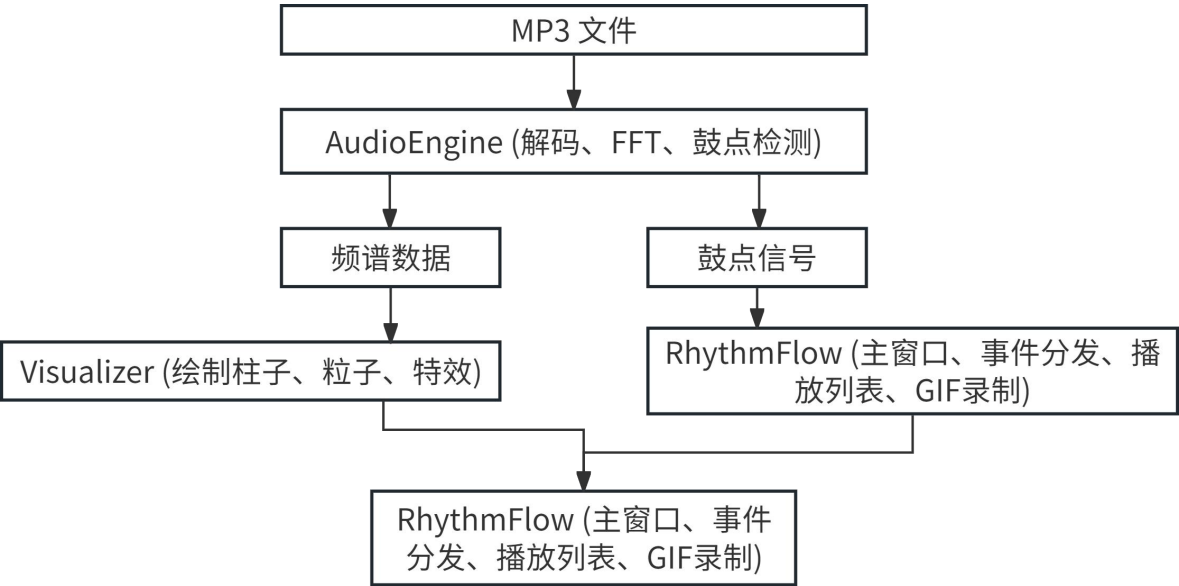
Visual Studio 2022+QT 6.11.0

三、课题要求

1. 面向对象
2. 图形化平台：使用 Qt 6.11 框架完成图形化界面开发，包含自定义绘制、透明窗口、动画特效等功能。
3. C++ 实现：全部核心逻辑使用 C++17 标准编写，包含面向对象设计、信号槽机制、多模块协作。
4. 版本迭代：项目在 Gitee/Github 上提交了多个版本，体现了从原型验证到功能完善、产品化打磨的完整开发过程。

四、主要流程

1. 整体架构



类名	职责
AudioEngine	MP3 播放、内部音频捕获、FFT 频谱计算、鼓点检测
Visualizer	所有图形绘制（频谱柱、粒子系统、涟漪、游戏界面、判定特效）
RhythmGame	游戏逻辑（音符生成、下落更新、按键判定、计分连击、连击加速）
RhythmFlow	主窗口控制（模块串联、事件分发、播放列表管理、GIF 录制）

2. 各模块实现

(1) 音频引擎模块 (AudioEngine)

① 主要函数:

- a. playFile(): 支持本地文件路径和 qrc 资源路径两种播放方式。

- b. `processAudioBuffer()`: 音频回调函数, 每次接收到音频数据时自动执行。

```
void AudioEngine::processAudioBuffer(const QAudioBuffer &buffer)

{

    const qint16 *data = buffer.constData<qint16>();

    int sampleCount = buffer.sampleCount();

    int channelCount = buffer.format().channelCount();

    // 填充 FFT 输入 (取第一声道, 归一化到[-1, 1])

    const int fftSize = 1024;

    std::vector<kiss_fft_cpx> in(fftSize), out(fftSize);

    for (int i = 0; i < fftSize && i * channelCount < sampleCount; ++i) {

        in[i].r = data[i * channelCount] / 32768.0f;

        in[i].i = 0.0f;

    }

    kiss_fft(m_fftCfg, in.data(), out.data());

    // 计算 64 频段对数能量

    for (int band = 0; band < BAR_COUNT; ++band) {

        // ... 频段划分、幅度计算、分贝转换 ...

        m_spectrum[band] = qMax(db + 80.0f, 0.0f);

    }

    emit spectrumUpdated();

}
```

- c. `togglePause()` / `isPaused()`: 播放控制。

② 关键技术:

- a. 使用 Qt6 的 QAudioBufferOutput 实现内部音频捕获，直接从解码器获取 PCM 数据，无需依赖麦克风。
- b. 集成轻量级 C 语言库 `kiss\_fft`，每次对 1024 个采样点进行 FFT，输出 64 个频段的能量值。
- c. 使用频谱通量法检测鼓点：计算当前帧与前一帧的频谱差异，当通量超过动态阈值时发射 beatDetected() 信号。

(2) 可视化绘制模块 (Visualizer)

① 主要功能:

- a. 频谱柱子：64 根青白渐变柱，使用 QLinearGradient 实现底部青色到顶部白色的过渡。柱高通过幂次变换增强对比度。
- b. 粒子系统：2000 个粒子，每个粒子具有向心力、切向力、低频扰动和运动轨迹。粒子使用 `QRadialGradient` 绘制为中心亮、边缘透明的柔光效果。
- c. 涟漪特效：鼠标左键点击时生成 5 圈淡金色同心圆环，从点击位置向外扩散并逐渐淡化消失。

// 鼠标点击时生成多圈涟漪

```
void Visualizer::mousePressEvent(QMouseEvent *event)
```

```
{
```

```
    int rippleCount = 5;
```

```
    float baseMaxRadius = 40.0f, radiusStep = 20.0f;
```

```
    for (int i = 0; i < rippleCount; ++i) {
```

```

        Ripple rip;

        rip.x = event->position().x();

        rip.y = event->position().y();

        rip.maxRadius = baseMaxRadius + i * radiusStep;

        rip.lifetime = 1.0f;

        m_ripples.push_back(rip);

    }

}

```

- d. 游戏界面：4 条轨道线（亮白霓虹光晕）、判定线（带动态反馈）、音符（PNG 素材）。

② 粒子物理核心逻辑（在 updateParticles() 中）：

```

void Visualizer::updateParticles()
{
    for (auto &p : m_particles) {

        float dx = m_currentAttractor.x() - p.x;

        float dy = m_currentAttractor.y() - p.y;

        float dist = sqrt(dx * dx + dy * dy);

        // 向心力 + 切向力

        float radialForce = attractStrength * (dist * 2.5f);

        float tangentialForce = tangentialBase

            * exp(-fabs(dist - idealDist) * 4.0f);

        p.vx += radialX * radialForce + tangentX * tangentialForce;
    }
}

```

```

        p.vy += radialY * radialForce + tangentY * tangentialForce;

        // 低频扰动

        if (bassNorm > 0.1f) {

            p.vx += rand() * 0.015f * bassNorm;

            p.vy += rand() * 0.015f * bassNorm;

        }

        // 阻尼 + 位置更新

        p.vx *= 0.97f; p.vy *= 0.97f;

        p.prevX = p.x; p.prevY = p.y; // 记录轨迹

        p.x += p.vx; p.y += p.vy;

    }

}

```

- a. 向心力：粒子受到指向引力中心（默认为屏幕中上部）的加速度。
- b. 切向力：垂直于径向的微小推力，产生旋转效果，峰值在理想环绕半径附近。
- c. 低频扰动：根据低频能量值向粒子施加随机推力，鼓点越强扰动越大。
- d. 运动轨迹：每帧记录粒子上一帧位置，绘制从旧位置到新位置的半透明线段。

(3) 节奏游戏模块 (RhythmGame)

① 主要函数：

- a. onBeatDetected(): 接收鼓点信号，在 4 条轨道中随机选择一条生成音符，10% 概率生成强拍音符。
- b. update(): 每帧更新音符下落位置，检测超出错过线的音符。

c. `pressKey()`: 按键判定函数, 查找对应轨道上距离判定线最近的音符。

② 关键设计:

- a. 自动谱面生成: 无需任何预制谱面文件, 所有音符由程序实时分析音频鼓点自动生成, 用户放入任意 MP3 都能获得专属游戏体验。
- b. 音符数据结构: 包含轨道编号、Y 坐标、击中/错过状态、强拍标记等字段。
- c. 连击计分: 基础分 100 分, 每 10 连击增加 1 分加成, 强拍音符额外加 50 分。

(4) 主窗口控制模块 (RhythmFlow)

① 主要功能:

- a. 模块串联: 创建 ``AudioEngine``、``Visualizer``、``RhythmGame`` 对象, 通过信号槽连接各模块。
- b. 事件过滤器: 重写 ``eventFilter()``, 统一处理所有按键事件 (数字键 1-4 控制播放器、空格键切换游戏模式、D/F/J/K 击打音符、R 键录制 GIF、F1 键显示帮助)。
- c. 播放列表管理: 维护 14 首内置歌曲的列表, 支持上一首/下一首切换和三种播放模式。
- d. GIF 录制: 集成单头文件库 ``gif.h``, 使用定时器每 83 毫秒抓取一帧可视化画面, 编码为 GIF 并保存到桌面。

3. 核心算法

(1) FFT 频谱分析算法

算法原理：快速傅里叶变换（FFT）将时域音频信号转换为频域能量分布。本项目使用 kiss_fft 库进行 1024 点实数 FFT，输出前 512 个复数值代表正频率分量。

计算流程：

- 从 QAudioBuffer 提取 16 位 PCM 数据，取第一个声道，归一化到 $[-1, 1]$ 。
- 填充 1024 点输入数组（不足补零）。
- 调用 `kiss_fft()` 执行变换。
- 计算每个频段的幅度值： $\text{mag} = \sqrt{\text{real}^2 + \text{imag}^2}$ 。
- 将 512 个频率点划分为 64 个对数频段，每个频段内取平均幅度。
- 转换为分贝值： $\text{dB} = 20 * \log_{10}(\text{avg} + 1e-6)$ ，并做动态范围调整。

(2) 鼓点检测算法

算法原理：频谱通量法（Spectral Flux）。鼓点通常表现为频谱能量的突然增加，通过计算前后两帧频谱的正差异和来检测。

计算公式：

$$\text{flux} = \sum \max(\text{spectrum}[i] - \text{prevSpectrum}[i], 0)$$

当 flux 超过历史平均通量的 1.25 倍且大于绝对阈值 0.8 时，判定为一次鼓点。

冷却机制：每次检测到鼓点后设置 8 帧冷却时间（约 200ms），防止同一鼓点被重复检测。

(3) 粒子物理系统核心公式

向心力： $F_{\text{radial}} = \text{attractStrength} \times \text{dist} \times 2.5$ ，方向指向引力中心。

切向力： $F_{\text{tangential}} = \text{tangentialBase} \times \exp(-|\text{dist} - \text{idealDist}| \times 4.0)$ ，方向垂直于径

向。

阻尼: $v *= 0.97$, 每帧速度衰减。

低频扰动: $\text{noise} = \text{rand}() \times 0.015 \times \text{bassNorm}$, 当低频能量大于阈值时施加。

(4) 连击加速机制

公式: $\text{speed} = \text{base} \times (1.0 + \text{comboLevel} \times 0.25)$

$\text{comboLevel} = \text{combo} / 25$, 即连击数每增加 25, 下落速度在基础速度上增加 25%。

基础速度为 1.25f (相对坐标/秒)。

(5) 音符判定算法

判定逻辑:

- 按键时, 遍历所有音符, 查找对应轨道上未被处理的音符。
- 计算音符 Y 坐标与判定线位置的距离 $\text{dist} = |\text{note.y} - \text{HIT_Y}|$ 。
- 选择距离最小的音符进行判定。
- 判定窗口为 ± 0.10 (相对坐标), Perfect 窗口为 ± 0.04 。
- 击中后标记 $\text{note.isHit} = \text{true}$, 计算分数并发射 noteHit 信号; 错过发射 noteMissed 信号。

// 按键判定逻辑

```
void RhythmGame::pressKey(int track)
```

```
{
```

```
    Note *bestNote = nullptr;
```

```
    float bestDist = HIT_WINDOW;    // 判定窗口  $\pm 0.10$ 
```

```

        for (auto &note : m_notes) {

            if (note.track == track && !note.isHit && !note.isMissed) {

                float dist = std::abs(note.y - HIT_Y);

                if (dist < bestDist) { bestDist = dist; bestNote = &note; }

            }

        }

        if (bestNote) {

            bestNote->isHit = true;

            int grade = (bestDist < PERFECT_WINDOW) ? 0 : 1;  // Perfect/Good

            emit noteHit(bestNote->track, bestNote->y, grade);

            m_combo++;

            // 计分逻辑...

        }

    }
}

```

五、测试与验证

本次对音频可视化节奏程序进行完整功能测试，包含启动界面、音频可视化、鼠标交互、播放控制、节奏游戏、录制功能等多项测试内容，所有测试项均测试通过，具体测试情况如下：

1. 启动画面测试：测试方式为双击运行程序，预期程序展示深青黑背景、旋转粒子以及青白渐变标题，程序启动约 2.5 秒后启动画面自动淡出，主窗口同步淡入，实际运行效果与预期一致，测试通过。

2. 频谱柱子测试：测试方式为播放任意 MP3 音频文件，预期 64 根频谱柱子跟随音乐节奏跳动，左侧低音区跳动幅度较大，右侧高音区幅度较小，且青白渐变渲染效果正常，实际运行符合预期，测试通过。
3. 安静状态粒子系统测试：在不播放音乐的状态下观察粒子效果，预期 2000 个粒子聚拢形成星云形态，缓慢旋转并附带微弱运动光尾，实际表现无误，测试通过。
4. 音乐状态粒子系统测试：播放鼓点特征明显的音乐，预期粒子跟随鼓点向外散开，低频爆发时粒子扰动效果增强，粒子整体保持青色系色调，实际效果达标，测试通过。
5. 鼠标点击聚拢测试：使用鼠标左键点击画面空白区域，预期粒子快速向点击位置聚拢，同时生成 5 圈淡金色涟漪向外扩散，粒子颜色渐变至亮白偏青色，交互效果正常，测试通过。
6. 鼠标拖动跟随测试：按住鼠标左键在画面中拖动，预期粒子持续跟随鼠标移动，且保留清晰拖动轨迹，实际交互流畅，测试通过。
7. 内置歌曲播放测试：点击播放按钮并在菜单中选择内置歌曲，预期程序正常播放歌曲，播放菜单展示 14 首中英文歌曲，实际功能正常，测试通过。
8. 本地文件播放测试：点击 MP3 按钮导入本地音频文件，预期音频正常播放，且本地文件不会添加至内置播放列表，实际运行符合设计要求，测试通过。
9. 数字按键播放控制测试：依次按下数字 1、2、3、4 按键，预期按键分别实现上一首、暂停/播放、下一首、播放模式切换功能，播放模式按照顺序、单曲、随机的规则循环切换，左下角功能提示文字可正常渐显渐隐，实际按键响应准确，测试通过。
10. 游戏模式切换测试：按下空格键，预期画面平滑过渡切换至游戏模式，界面正常显示 4 条音符轨道与判定线，切换流畅无卡顿，测试通过。

11. 音符生成测试：在游戏模式下播放音乐，预期程序根据鼓点自动生成并下落音符，无漏检、误检异常情况，音符生成逻辑正常，测试通过。
12. 按键判定测试：通过 D、F、J、K 按键击打下落音符，命中音符时界面显示 Perfect 或 Good 判定，判定线短暂亮白加粗；未命中音符时显示 Miss 判定，判定线灰色闪烁，判定机制准确，测试通过。
13. 连击加速测试：游玩过程中累积连击数，预期连击数超过 25 时音符下落速度加快，每累积 25 次连击速度递增 25%，连击中断后速度恢复初始基础速度，加速规则执行正常，测试通过。
14. GIF 录制测试：按下 R 键开启录制，预期界面左下角显示录制提示文字，录制时长为 5 秒，录制结束后自动在桌面生成带时间戳命名的 GIF 文件，录制功能正常，测试通过。
15. 帮助界面测试：按下 F1 功能键，预期弹出帮助对话框，对话框内展示完整的程序操作说明，弹窗功能正常，测试通过。
16. 程序退出测试：鼠标右键点击窗口并选择退出选项，预期程序无卡顿、无报错正常关闭，退出流程安全稳定，测试通过。

六、收获与总结

1. 技术收获

(1) 对 Qt 图形框架的深入理解

在开发过程中，我深入学习了 Qt6 的核心机制。QPainter 的绘制管线、信号槽的异步通信、QAudioBufferOutput 的内部音频捕获，这些知识的实践让我对一个完整的图形化程序有了更深刻的理解。特别是在实现透明悬浮窗和外发光效果时，

Qt::FramelessWindowHint 和 QGraphicsDropShadowEffect 的组合使用让我认识到 Qt 提供的强大定制能力。

(2) 第三方库的集成与封装

项目中集成了两个第三方库：`kiss\_fft`（FFT 频谱分析）和 `gif.h`（GIF 编码）。在这个过程中，我学会了如何将 C 语言库通过 `extern "C"` 引入 C++ 项目，以及如何将单头文件库封装为独立的模块类（如 `GifRecorder`）以避免头文件冲突。这种“识别需求→寻找轻量级库→封装集成”的模式，是实际工程中非常实用的技能。

(3) 模块化架构设计

将项目拆分为 `AudioEngine`、`Visualizer`、`RhythmGame`、`RhythmFlow` 四个独立模块，每个模块只负责单一职责，通过 Qt 信号槽进行解耦通信。这种设计带来的好处在项目后期尤为明显——当我需要添加 GIF 录制、内置歌单管理等功能时，只需修改 `RhythmFlow` 类，而不需要触碰音频引擎或可视化绘制的代码。

(4) 调试能力与问题排查

在开发中遇到了许多问题：`gif.h` 头文件重定义、中文编码导致 C2001 错误、QPropertyAnimation 动画结束后窗口变透明、windeployqt 打包后缺少 FFmpeg 依赖等。每一次问题的排查和解决，都让我对 C++ 编译机制、Qt 动画系统、Windows 部署流程有了更深的理解。

2. AI 说明

① 使用模型

本项目在开发过程中，使用了以下两个大语言模型作为辅助开发工具：

- a. DeepSeek V4.0 深度求索
- b. Kimi 2.6 月之暗面

② 辅助内容与方式

AI 工具在本项目中主要介入以下方面：

- a. 项目初期，本人就项目的技术选型、模块划分、数据流设计等问题与模型进行了多轮讨论。模型提供了 Qt 框架下音频捕获的方案比较、FFT 库的选型建议以及粒子系统的物理模型参考。
- b. 编码阶段，模型为部分样板代码的生成提供了参考，例如 QAudioBufferOutput 的初始化配置、kiss_fft 库的调用封装、GIF 录制帧循环的骨架代码等。
- c. 调试阶段，当遇到编译错误或运行时异常时，将错误信息和上下文提交给模型，获取可能的原因分析和修复方向的建议。例如 gif.h 的头文件包含冲突问题、中文编码导致的 C2001 错误、QPropertyAnimation 结束后窗口透明度异常等问题，模型的建议缩小了排查范围。

3. 总述

RhythmFlow 不仅是一次大作业，更是一次完整的实践。它让我真切地体会到：，写一个“好看又好用”的程序，需要投入大量的心血。当我在 B 站上看到陌生人的点赞和评论时，那种被认可的感觉让一切努力都变得值得。这次经历将成为我编程道路上最宝贵的记忆之一。